Basilisk 101: Computational Fluid Dynamics with Basilisk C

Computational Multiphase Flows with Basilisk: From Champagne Bubbles to Drop Impact

Vatsal Sanjay

Web version

TL;DR

This tutorial explores computational approaches to multiphase flow phenomena using Basilisk's axisymmetric framework. We examine five progressively complex simulations: bubble bursting, drop impact on pools, drop impact on solid surfaces, liquid lens equilibrium, and heated drop impact. Each simulation incorporates different physical aspects – surface tension, inertia, viscosity, thermal effects – characterized by dimensionless parameters including Ohnesorge, Bond, Weber, and Froude numbers. Through these simulations, we develop practical skills in handling complex interfaces, implementing boundary conditions, and visualizing flow features using adaptive mesh refinement techniques specifically designed for multiphase problems.

Introduction

In this tutorial, we explore multiphase flow phenomena through computational simulations using the Basilisk framework. Multiphase flows – involving interactions between different fluids or phases – represent some of the most complex and visually striking phenomena in fluid dynamics. From bursting bubbles in champagne to raindrops impacting puddles, these phenomena are ubiquitous in both natural processes and industrial applications.

Building on our previous work with heat conduction (1st-workingAssignment) and single-phase flows (2nd-workingAssignment), we now tackle the additional complexity introduced by fluid interfaces, surface tension, and multiple interacting phases.

Learning Objectives

- Implement and analyze axisymmetric multiphase flow simulations
- Understand key dimensionless parameters governing multiphase dynamics
- Develop skills in handling complex interfaces and boundary conditions
- Visualize and interpret dynamic interface evolution
- Apply adaptive mesh refinement strategies for multiphase problems
- Extend simulations to incorporate thermal effects

Throughout this tutorial, we'll progress through five core simulation cases:

1. Bubble bursting at a free surface
2. Drop impact on a liquid pool
3. Drop impact on a solid surface
4. Equilibrium of liquid lenses
5. Heated drop impact on a solid surface

Each case introduces new physical aspects and numerical challenges, providing a comprehensive overview of computational multiphase fluid dynamics.

Part 1: Drop Impact Simulations

1.1 Drop Impact on a Liquid Pool (3-DropImpactOnPool.c)

When a liquid drop impacts a pool of the same liquid, complex dynamics unfold including crater formation, capillary wave propagation, crown formation, and potential splashing or bubble entrapment.

Problem Statement

Simulate the axisymmetric impact of a liquid drop onto a pool of the same liquid. Key dimensionless parameters include:

- Froude number (Fr): $\text{Fr} = \frac{U^2}{gD}$ Ratio of inertial to gravitational forces
- Galilei number (Ga): $\text{Ga} = \frac{\rho g D^3}{\mu^2}$ Ratio of gravitational to viscous forces
- Bond number (Bo): $\text{Bo} = \frac{\rho g D^2}{\sigma}$ Ratio of gravitational to surface tension forces

Where U is impact velocity, D is drop diameter, g is gravity, μ is viscosity, ρ is density, and σ is surface tension.

The implementation includes several advanced techniques:

```
#include "axi.h"                      // Axisymmetric coordinates
#include "navier-stokes/centered.h"   // NS solver with centered discretization
#define FILTERED 1                    // Enable filtered VOF for stability
#include "two-phase.h"                // Two-phase flow solver
#include "navier-stokes/conserving.h" // Conservative momentum advection
#include "tension.h"                  // Surface tension forces
#include "reduced.h"                  // Reduced gravity formulation
```

A key feature of this simulation is the use of a tracer field to track the drop liquid even after it merges with the pool:

```
scalar tagDrop[];                     // Tracer to identify the drop liquid
event init(t = 0) {
  if (!restore(file = "dump")){
    // Initialize the fluid configuration:
    // - Combined shape of drop and pool using union of:
    //   - Drop: circle centered at (Hint,0) with radius 1
    //   - Pool: half-plane where x < 0
    fraction(f, union(1. - sq(x-Hint) - sq(y), -x));

    // Initialize the tracer to track only the drop (not the pool)
    fraction(tagDrop, 1 - sq(x-Hint) - sq(y));

    // Add the drop tracer to the list of fields advected with f
    f.tracers = list_append(f.tracers, tagDrop);
```

```
    // Set initial downward velocity for the drop only
    foreach(){
      u.x[] = -tagDrop[]*sqrt(Fr);    // Velocity proportional to √Fr
    }
  }
}
```

Reduced Gravity Formulation

The simulation uses the "reduced.h" module, which implements a more efficient approach to handling pressure in the presence of gravity. Instead of solving for the total pressure, it solves for the reduced pressure: $p' = p - \rho g z$. This approach provides better numerical stability for low-Froude-number flows.

1.2 Drop Impact on Solid Surfaces (3-DropImpactOnSolids.c)

Drop impact on solid surfaces exhibits different dynamics compared to pool impact, with spreading, possible rebound, and various splashing behaviors depending on the parameters.

Problem Statement

Simulate the axisymmetric impact of a liquid drop on a solid surface. Key dimensionless parameters include:

- Weber number (We): $\mathrm{We} = \frac{\rho U^2 D}{\sigma}$ Ratio of inertial to surface tension forces
- Ohnesorge number (Oh): $\mathrm{Oh} = \frac{\mu}{\sqrt{\rho \sigma D}}$ Ratio of viscous forces to inertial and surface tension forces
- Bond number (Bo): $\mathrm{Bo} = \frac{\rho g D^2}{\sigma}$ Ratio of gravitational to surface tension forces

The boundary conditions for the solid wall are implemented as:

```
// Left boundary: solid wall (no-slip and no flux)
u.t[left] = dirichlet(0.0);     // Tangential velocity = 0 (no-slip)
f[left] = dirichlet(0.0);       // Volume fraction = 0 (solid wall)

// Right boundary: outflow condition
u.n[right] = neumann(0.);       // Zero gradient for normal velocity
p[right] = dirichlet(0.0);      // Reference pressure = 0
```

The drop is initialized with a downward velocity and positioned slightly above the wall:

```
event init(t = 0){
  if(!restore(file = "dump")){
    // Refine mesh near the drop interface
    refine((R2Drop(x,y) < 1.05) && (level < MAXlevel));

    // Initialize the drop shape using volume fraction
    fraction(f, 1. - R2Drop(x,y));

    // Set initial velocity field (drop approaching the surface)
    foreach () {
      u.x[] = -1.0*f[];  // Initial velocity in x-direction (toward wall)
```

```
      u.y[] = 0.0;       // No initial velocity in y-direction
    }
  }
}
```

Dissipation-Based Refinement

This simulation includes an advanced refinement criterion based on the local dissipation rate:

```
foreach(){
  // Calculate velocity gradient components in cylindrical coordinates
  double D11 = (u.y[0,1] - u.y[0,-1])/(2*Delta);
  double D22 = (u.y[]/max(y,1e-20));
  double D33 = (u.x[1,0] - u.x[-1,0])/(2*Delta);
  double D13 = 0.5*((u.y[1,0] - u.y[-1,0] + u.x[0,1] - u.x[0,-1])/(2*Delta));

  // Calculate dissipation rate (sum of squares of strain rates)
  double D2 = (sq(D11)+sq(D22)+sq(D33)+2.0*sq(D13));
  D2c[] = f[]*D2;  // Dissipation rate in the drop phase
}
```

This ensures that regions with high energy dissipation (typically near the spreading front and stagnation point) are well-resolved.

1.3 Heated Drop Impact (3-HeatedDropImpact.c)

This advanced simulation extends the drop impact problem to include thermal effects, modeling the heat transfer that occurs when a cold drop impacts a heated surface.

Problem Statement

Simulate the thermal-fluid dynamics of a cold liquid drop impacting a heated solid surface. In addition to the mechanical parameters (We, Oh, Bo), the simulation includes thermal parameters:

- Prandtl number ($\mathrm{Pr} = \nu/\alpha$): Ratio of momentum diffusivity to thermal diffusivity
- Thermal diffusivity ratio (D1/D2): Ratio of thermal diffusivities between the phases

The simulation solves both the Navier-Stokes equations and the heat transfer equation:

```
#include "axi.h"                // Axisymmetric coordinates
#include "navier-stokes/centered.h"  // NS solver with centered discretization
#define FILTERED                // Use filtered VOF advection
#include "two-phase-thermal.h"  // Two-phase flow with heat transfer
#include "navier-stokes/conserving.h"  // Conservative momentum advection
#include "tension.h"            // Surface tension model
#include "reduced.h"            // Reduced gravity approach
```

The thermal boundary conditions specify a constant heat flux from the hot substrate to the cold drop:

```
// Thermal boundary condition: constant heat flux from the hot substrate
T[left] = neumann(10.0);        // Fixed positive temperature gradient (heating the cold drop)
```

Physical properties include thermal diffusivity for both phases:

```
// Drop phase (1) properties
rho1 = 1.0;                    // Density of drop (reference value)
mu1 = Ohd/sqrt(We);            // Viscosity derived from Ohnesorge number
D1 = 1.0;                      // Thermal diffusivity of drop (reference value)

// Surrounding phase (2) properties
rho2 = Rho21;                  // Density ratio (air/water)
mu2 = Ohs/sqrt(We);            // Viscosity derived from Ohnesorge number
D2 = 1e-3;                     // Thermal diffusivity ratio (air/water ~ 10^-3)
```

Thermal Gradients and Adaptive Refinement The adaptive mesh refinement criteria now include temperature gradients alongside interface position, curvature, and velocity gradients:

```
adapt_wavelet((scalar *){f, KAPPA, u.x, u.y, D2c, T},
              (double[]){fErr, KErr, VelErr, VelErr, DissErr, TErr},
              MAXlevel, MINlevel);
```

This ensures that thermal boundary layers and temperature gradients are properly resolved.

Part 2: Bubble Bursting Dynamics

2.2 Physical Background

When a bubble reaches a free surface, a thin film separates it from the surrounding atmosphere. This film eventually ruptures, leading to capillary waves traveling along the interface, followed by the collapse of the cavity and potentially the ejection of droplets. This phenomenon is relevant to numerous natural and industrial processes:

- Aerosol production from ocean waves
- Champagne bubbles and carbonated beverages
- Volcanic eruptions
- Industrial foaming processes

Key Physical Parameters Two dimensionless numbers primarily govern bubble bursting dynamics:

- Ohnesorge number (Oh): $\mathrm{Oh} = \frac{\mu}{\sqrt{\rho\sigma R}}$ Relates viscous forces to inertial and surface tension forces
- Bond number (Bo): $\mathrm{Bo} = \frac{\rho g R^2}{\sigma}$ Relates gravitational forces to surface tension forces

Where μ is viscosity, ρ is density, σ is surface tension, R is bubble radius, and g is gravitational acceleration.

2.2 Implementing Bubble Bursting (3-BurstingBubbles.c)

The simulation uses Basilisk's axisymmetric framework to model the bursting process:

```
#include "axi.h"                      // Axisymmetric coordinates
#include "navier-stokes/centered.h"   // NS solver with centered discretization
#define FILTERED                      // Smear density and viscosity jumps for stability
#include "two-phase.h"                // Two-phase flow solver
#include "navier-stokes/conserving.h" // Conservative momentum advection
#include "tension.h"                  // Surface tension forces
```

The bubble is initially defined using a pre-calculated equilibrium shape (based on the Bond number) that balances surface tension and gravitational forces:

```
event init (t = 0) {
  if (!restore (file = dumpFile)){
    // Read initial shape from data file
    char filename[60];
    sprintf(filename,"Bo%5.4f.dat",Bond);
    FILE * fp = fopen(filename,"rb");

    // Generate distance field from shape coordinates
    coord* InitialShape;
    InitialShape = input_xy(fp);
    fclose (fp);

    scalar d[];
    distance (d, InitialShape);

    // Refine mesh around interface
    while (adapt_wavelet ((scalar *){f, d}, (double[]){1e-8, 1e-8}, MAXlevel).nf);

    // Convert distance field to volume fraction
    vertex scalar phi[];
    foreach_vertex(){
      phi[] = -(d[] + d[-1] + d[0,-1] + d[-1,-1])/4.;
    }
    fractions (phi, f);
  }
}
```

Adaptive Mesh Refinement The simulation uses multiple criteria for mesh refinement to accurately resolve the interface dynamics:

```
event adapt(i++){
  // Calculate interface curvature for refinement criterion
  scalar KAPPA[];
  curvature(f, KAPPA);

  // Refine mesh based on multiple criteria
  adapt_wavelet ((scalar *){f, u.x, u.y, KAPPA},
                 (double[]){fErr, VelErr, VelErr, KErr},
                 MAXlevel, MAXlevel-6);
}
```

This adaptive approach concentrates computational resources where they're needed most: at the interface, in regions of high curvature, and in areas with significant velocity gradients.

The simulation tracks the kinetic energy to monitor the dynamics and determine when the system has reached a quasi-steady state:

```
event logWriting (i++) {
  // Calculate total kinetic energy (with axisymmetric integration)
  double ke = 0.;
  foreach (reduction(+:ke)){
    ke += (2*pi*y)*(0.5*rho(f[])*(sq(u.x[]) + sq(u.y[])))*sq(Delta);
  }

  // Output to log and check for convergence/divergence
  fprintf (ferr, "%d %g %g %g\n", i, dt, t, ke);

  // Check for energy explosion (instability)
  if (ke > 1e2 && i > 1e1){
    fprintf(ferr, "The kinetic energy blew up. Stopping simulation\n");
    return 1;
  }

  // Check for near-static conditions (completion)
  if (ke < 1e-6 && i > 1e1){
    fprintf(ferr, "kinetic energy too small now! Stopping!\n");
    return 1;
  }
}
```

2.3 Expected Results

For low Ohnesorge numbers (low viscosity), we expect to see:

1. Rapid retraction of the film after rupture
2. Capillary waves traveling along the interface
3. Formation of a central jet (Worthington jet)
4. Possible pinch-off of droplets from the jet

For higher Ohnesorge numbers (higher viscosity), viscous damping suppresses these features, resulting in a more gradual collapse of the cavity.

Part 3: Liquid Lens Equilibrium

3.1 Three-Phase Systems (3-EquilibriumOfLiquidLenses.c)

A liquid lens forms when a drop of one liquid sits at the interface between another liquid and a gas. The equilibrium shape is determined by the balance of surface tension forces at the three-phase contact line.

Problem Statement S

imulate the equilibrium shape of a liquid lens at the interface between two immiscible fluids. The system consists of three phases:

1. Dispersed phase (labeled as phase 1)
2. Water drop (labeled as phase 2)
3. Air (labeled as phase 3)

Surface tension coefficients between the three pairs of phases determine the equilibrium contact angles according to the Neumann triangle condition.

This simulation uses Basilisk's three-phase flow solver:

```
#include "axi.h"                                          // Axisymmetric coordinates
#include "navier-stokes/centered.h"                       // Centered finite-volume
 ↪  Navier-Stokes solver
#define FILTERED                                         // Enable property filtering for
 ↪  stability
#include "three-phase.h"                                 // Three-phase interface tracking
#include "tension.h"                                     // Surface tension model
```

Surface tension coefficients are defined between each pair of phases:

```
// Set surface tension coefficients between phase pairs
f1.sigma = 1.0;                       // Surface tension: dispersed phase-air (σ₁₃, reference
 ↪  value)
f2.sigma = 52/16;                     // Surface tension: dispersed phase-water drop (σ₁₂ ≈
 ↪  3.25)

// Note: The effective surface tension between water and air (σ₂₃) will be:
// σ₂₃ = σ₁₃ + σ₁₂ = 1.0 + 52/16 = 4.25
// This satisfies the ideal precursor film assumption (Neumann triangle)
```

The initial configuration puts a water drop slightly below the interface between the dispersed phase and air:

```
event init(t = 0){
  if(!restore (file = "dump")){
    // Refine mesh around the water drop for better resolution
    refine(sq(x+1.025) + sq(y) < sq(1.2) && level < MAXlevel);

    // Initialize the water drop (phase 2) as a circle slightly below the interface
    // Circle equation: (x+1.025)² + y² = 1, with radius=1 and center at (-1.025,0)
    fraction(f2, 1.0 - sq(x+1.025) - sq(y));

    // Initialize the dispersed phase (phase 1) in the left half of the domain
    // x<0 is dispersed phase, x>0 is air
    fraction(f1, -x);

    // Update the boundary conditions
    boundary ((scalar *){f1, f2});
  }
}
```

Neumann Triangle and Contact Angles The equilibrium contact angles at the three-phase junction are determined by the balance of surface tension forces, known as the Neumann triangle condition:

$$\frac{\sigma_{12}}{\sin\theta_3} = \frac{\sigma_{23}}{\sin\theta_1} = \frac{\sigma_{13}}{\sin\theta_2} \tag{1}$$

Where σ_{ij} is the surface tension between phases i and j, and θ_k is the angle in phase k.
In this simulation, we've set $\sigma_{13} = 1.0$ and $\sigma_{12} = 3.25$, resulting in specific equilibrium contact angles at the three-phase junction.

Part 4: Exercises and Extensions

Now that we've explored the fundamentals of these multiphase flow simulations, let's develop your skills further with some practical exercises.

4.1 Bubble Bursting Exercise

Exercise: Investigating Ohnesorge Number Effects

Modify the bubble bursting simulation to explore different Ohnesorge numbers:

1. Run simulations with Oh = 1e-3, 1e-2, and 1e-1, keeping Bo constant
2. Track and compare the following:
    - Maximum jet height
    - Time to first droplet pinch-off (if it occurs)
    - Maximum kinetic energy during the process
3. Create plots showing these metrics as functions of Oh
4. Explain the physical mechanisms behind the observed trends

This exercise will help you understand how viscosity affects capillary-inertial dynamics.

4.2 Drop Impact Exercise

Exercise: Comparing Pool and Solid Surface Impact

Conduct a comparative study between drop impact on a pool and on a solid surface:

1. Set up equivalent simulations for both scenarios:
    - Use the same Weber number and Ohnesorge number
    - Match other parameters (domain size, resolution, etc.)
2. Track and compare the following:
    - Maximum spread diameter
    - Crater depth (for pool impact)
    - Energy dissipation rate
3. Visualize the differences in flow patterns using streamlines or velocity magnitude contours
4. Explain why the outcomes differ based on the underlying physics

Hint: Pay special attention to the boundary conditions and how they affect momentum transfer.

4.3 Advanced Exercise: Thermal Effects

Exercise: Temperature-Dependent Surface Tension

Modify the heated drop impact simulation to include temperature-dependent surface tension:

1. Implement the linear relationship: $\sigma(T) = \sigma_0(1 - \gamma(T - T_0))$
   - σ_0 is the reference surface tension at temperature T_0
   - γ is the temperature coefficient (typically positive)
2. Run simulations with $\gamma = 0$ (constant surface tension) and $\gamma = 0.01$ (temperature-dependent)
3. Analyze how Marangoni effects (surface tension gradients) affect the spreading dynamics
4. Visualize temperature and velocity fields to identify regions where thermal effects are most significant

This extension introduces thermocapillary phenomena, which are important in many industrial processes.

Part 5: Implementation Tips and Best Practices

Based on our exploration of these multiphase simulations, here are some key insights and best practices:

Multiphase Simulation Tips

1. Interface resolution: Always ensure sufficient resolution near interfaces, especially in regions of high curvature. The `adapt_wavelet()` function with appropriate error thresholds is crucial.
2. Numerical stability: For high-density and viscosity ratios (typical in air-water systems), use:
   - The `FILTERED` option to smooth property jumps
   - The `navier-stokes/conserving.h` module for better momentum conservation
   - Appropriate timestep limitations (CFL < 0.5 is often necessary)
3. Boundary conditions: Pay careful attention to boundary conditions, especially for solid surfaces. Use `dirichlet()` and `neumann()` appropriately.
4. Initial conditions: Start with a well-resolved and physically reasonable initial condition. For drop impact, position the drop slightly above the surface.
5. Convergence monitoring: Track relevant physical quantities (kinetic energy, interface position) to determine when the simulation has reached a quasi-steady state.
6. Parameter sensitivity: Multiphase flows can be sensitive to small parameter changes. Conduct parameter sweeps to understand the system behavior.

Dimensionless Parameters

Understanding the key dimensionless parameters is essential for characterizing multiphase flows:

- Weber number (We): Inertia vs. surface tension
- Ohnesorge number (Oh): Viscosity vs. inertia and surface tension
- Bond number (Bo): Gravity vs. surface tension
- Froude number (Fr): Inertia vs. gravity
- Capillary number (Ca): Viscosity vs. surface tension

These parameters form a framework for classifying and predicting multiphase flow behavior across different scales and fluid properties.

Conclusion

In this tutorial, we've explored a range of fascinating multiphase flow phenomena through computational simulations using Basilisk. From bubble bursting to heated drop impact, these simulations capture complex interfacial dynamics that would be challenging to study experimentally.

Key insights from our exploration include:

1. The importance of surface tension in driving capillary-inertial phenomena
2. The role of viscosity in damping interfacial oscillations and jet formation
3. The complex interplay between solid boundaries and fluid interfaces
4. The additional complexity introduced by thermal effects
5. The power of adaptive mesh refinement in efficiently resolving multiscale features

These multiphase flow simulations demonstrate the capability of modern computational methods to capture complex physics with remarkable detail, providing insights into phenomena that occur too quickly or at scales too small for easy experimental observation.

Further Exploration

Consider these questions for deeper investigation:

1. How would adding surfactants affect bubble bursting or drop impact dynamics?
2. What happens when the drop and pool liquids have different properties (e.g., immiscible liquids)?
3. How do contact angle effects influence drop impact on solid surfaces?
4. What role does the ambient gas play in drop impact phenomena?
5. How would phase change (evaporation or solidification) alter these dynamics?

Metadata

Author:: Vatsal Sanjay Date published:: March 12, 2025

Date modified:: March 12, 2025 at 09:30 CET

Back to main website

Home, Team, Research, Github